

Visual Documentation

Architecture Diagrams & Project Screenshots

Doctor Clinic DevOps System · Document 07

Muhammed Munir Al Tawil · DevOps · DataScientest / Sorbonne University · May 2026

1. System Architecture Diagrams

Figure 1 – High-Level Architecture

The Doctor Clinic DevOps System is organized into 5 layers: users, Docker Compose runtime (Nginx + Laravel + MySQL), Proxmox VE deployment with NAT routing, CI/CD automation via GitLab, and operations support (backup, monitoring, Terraform IaC).

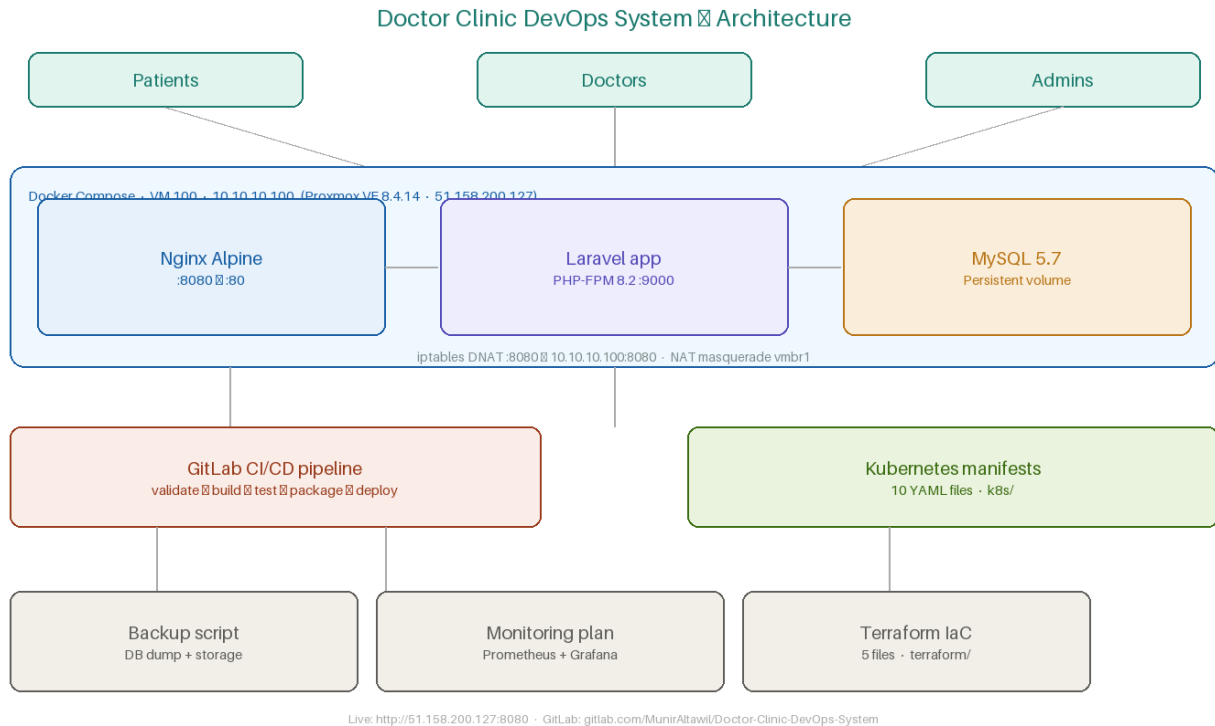


Figure 1. Doctor Clinic DevOps System – Full architecture overview

Figure 2 – CI/CD Pipeline Flow

The GitLab CI/CD pipeline runs 5 sequential stages automatically on every push to the main branch. PHP syntax validation ensures code quality before Docker images are used for building, testing, packaging, and deployment.

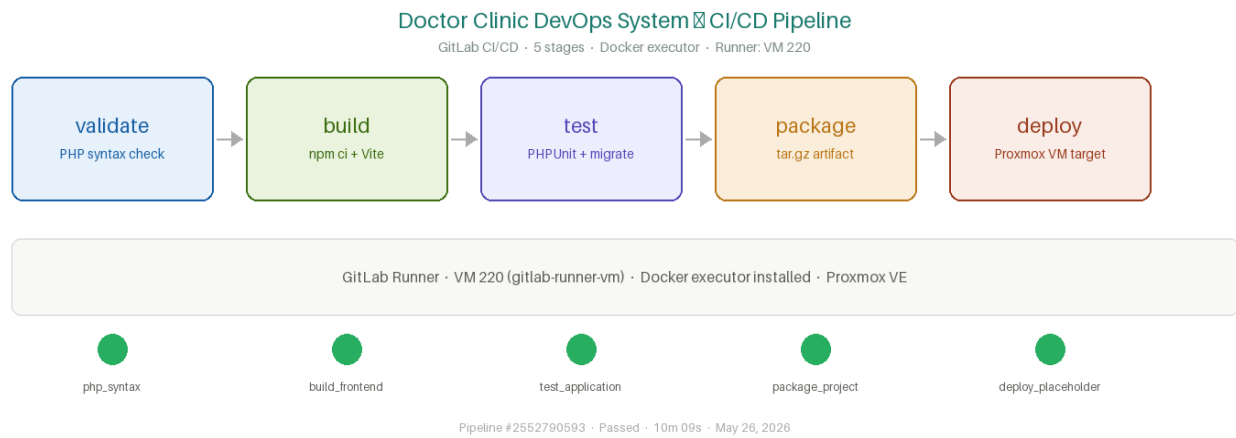


Figure 2. GitLab CI/CD pipeline – 5 stages, Docker executor, VM 220 runner

Figure 3 – Proxmox Network Architecture

The application is deployed on VM 100 on an internal bridge (vmbr1, 10.10.10.0/24) with NAT masquerade to the external bridge (vmbr0, 51.158.200.127/24). iptables DNAT rules forward port 8080 from the public IP to the VM's Nginx container.

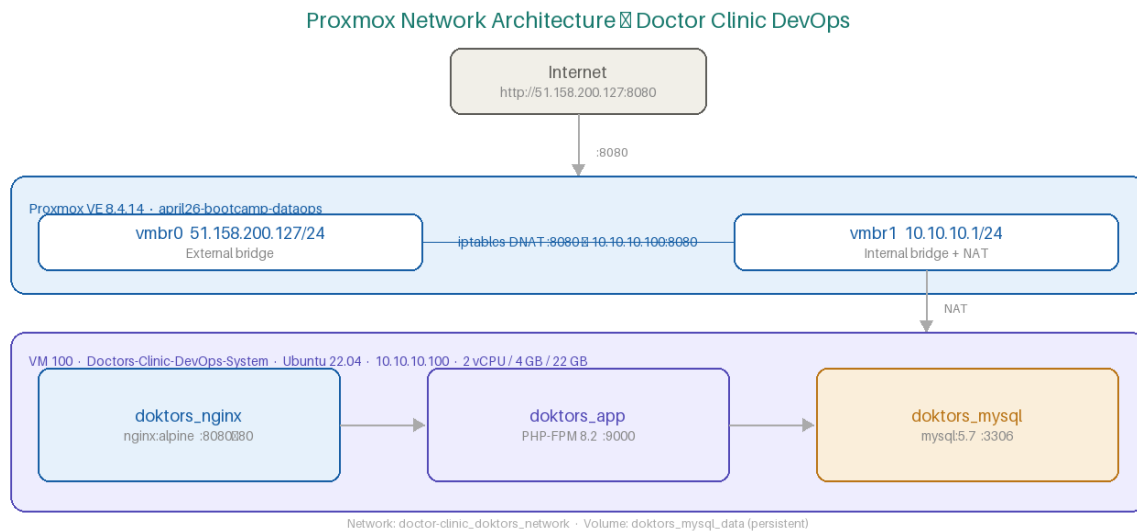


Figure 3. Proxmox VE network architecture – NAT, vmbr0/vmbr1, port forwarding

2. Live Application Screenshots

Figure 4 – Live Application Homepage

The clinic platform is publicly accessible at <http://51.158.200.127:8080>. The homepage presents navigation (Doctors, Specialties, About, Contact, Login, Register), a hero section, and live rhythm feature cards. Deployed and verified on May 26, 2026.

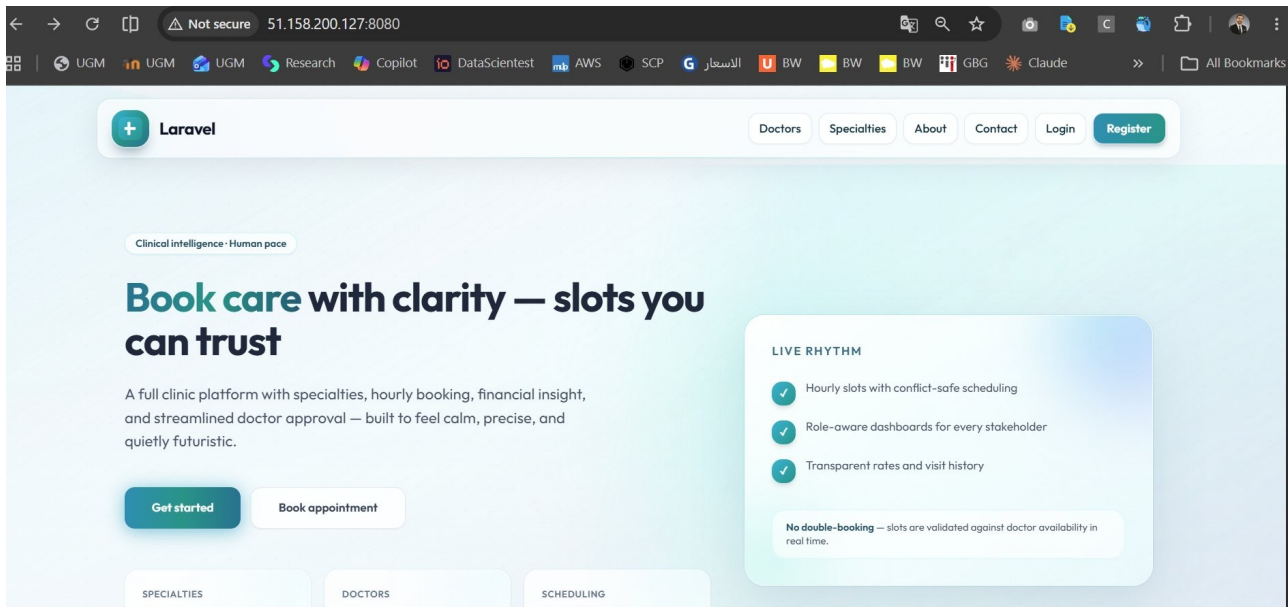


Figure 4. Live application at <http://51.158.200.127:8080> – May 26, 2026

Figure 5 – Doctor Dashboard

The Doctor dashboard shows live clinical data after seeding: 248 appointments, 57,970€ in earnings, and 10,230€ in clinic fees retained. All user roles (patient, doctor, admin) are fully functional with demo data.

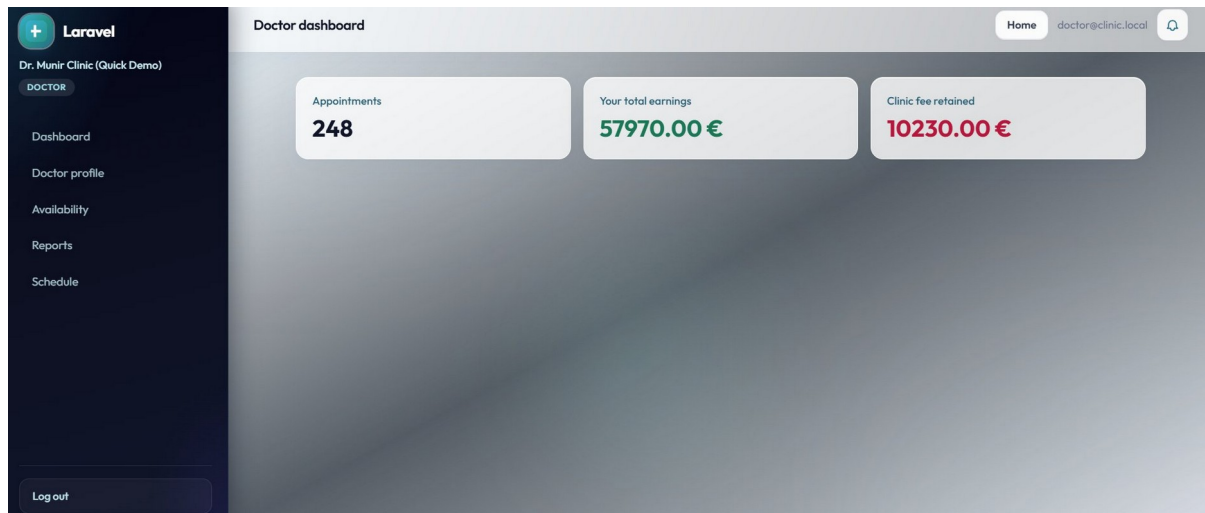


Figure 5. Doctor dashboard – live demo data with Dr. Munir Clinic (Quick Demo)

3. CI/CD Pipeline Screenshots

Figure 6 – GitLab Pipelines List (All Passed)

Both pipelines show Passed status with all 5 stages completing successfully. Pipeline #2552790593 ran in 10 minutes 9 seconds and pipeline #2552622134 ran in 8 minutes 33 seconds, demonstrating reliable automated delivery.

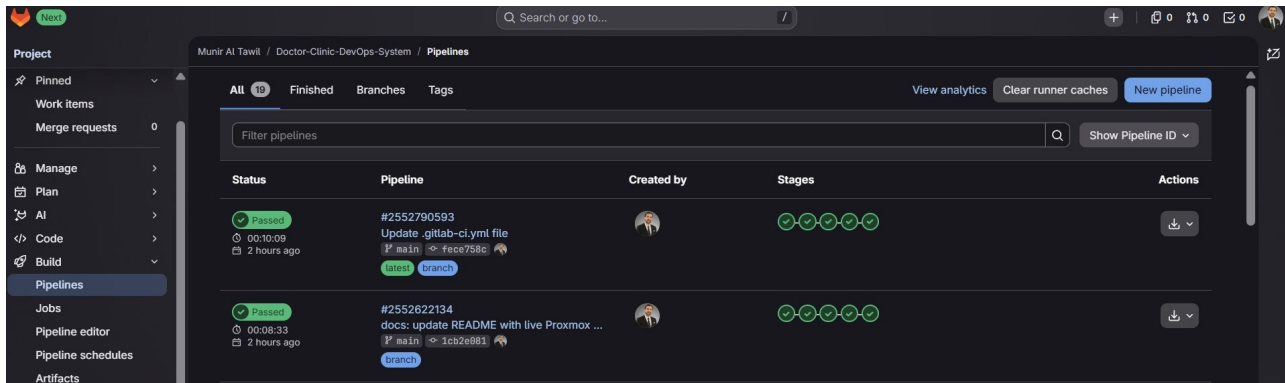


Figure 6. GitLab CI/CD pipelines – Passed status, 5 stages each, May 26, 2026

Figure 7 – GitLab Jobs (php_syntax Passed)

The `php_syntax` validation job passed on both pipelines, confirming that all PHP source files in the application are syntactically valid. Jobs run inside a PHP 8.2 Docker container on the self-hosted runner.

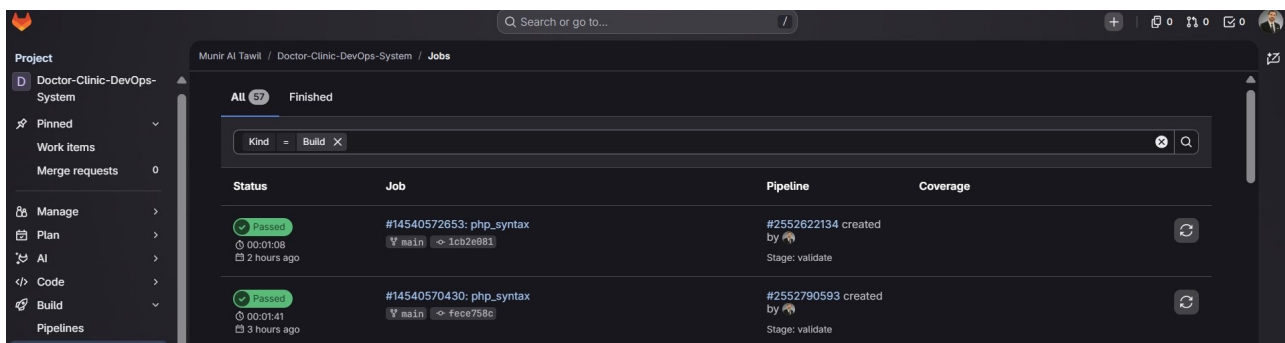


Figure 7. GitLab jobs view – `php_syntax` passed on both pipeline runs

4. Proxmox Infrastructure Screenshots

Figure 8 – Proxmox VE with VM 100 and VM 220

The Proxmox VE 8.4.14 node hosts two VMs: VM 100 (Doctors-Clinic-DevOps-System) running the application stack, and VM 220 (gitlab-runner-vm) providing the CI/CD runner. Docker was installed on VM 220 to enable the Docker executor.

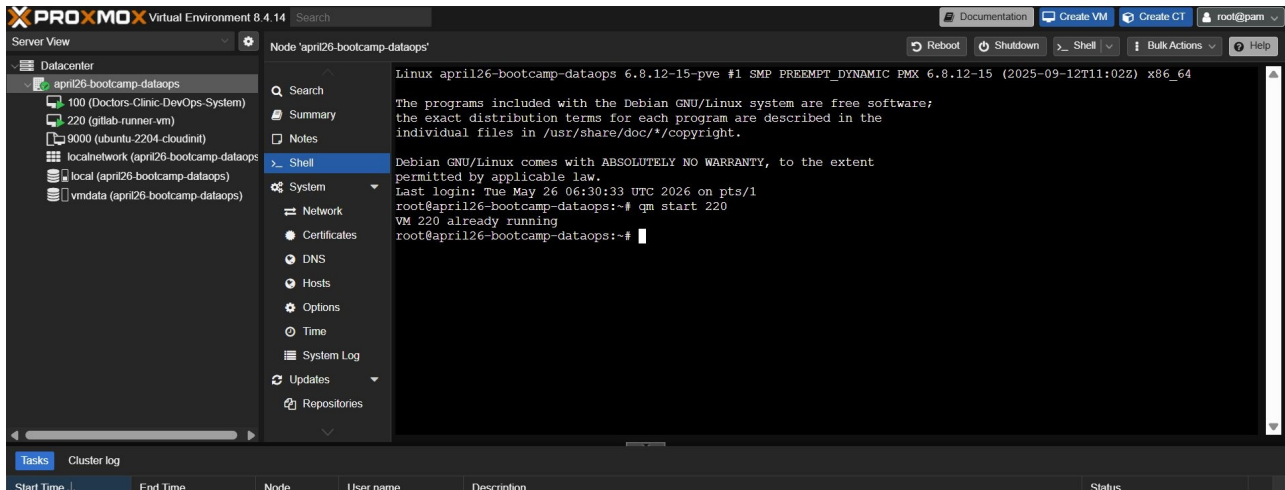


Figure 8. Proxmox VE – VM 100 (app) and VM 220 (runner) both running

Figure 9 – Docker installation on GitLab Runner VM

Docker CE was installed on VM 220 to enable the GitLab Runner Docker executor. The runner was registered with the GitLab project and configured to use Docker images (php:8.2, node:20, alpine) for isolated job execution.

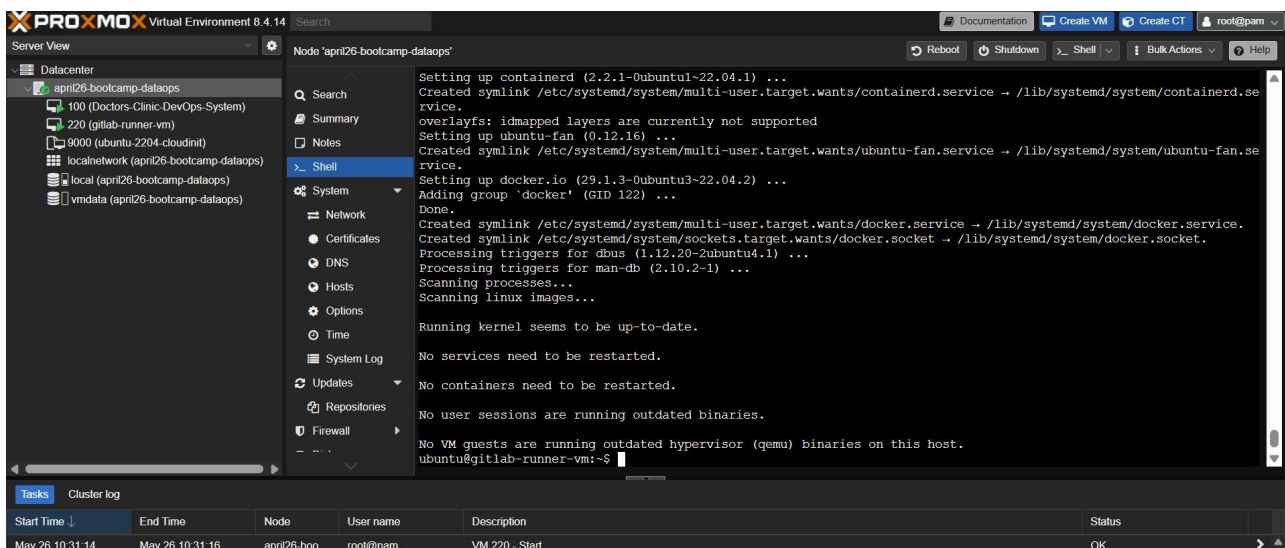


Figure 9. Docker installation on VM 220 (gitlab-runner-vm) – May 26, 2026

5. Source Control Screenshots

Figure 10 – GitHub Repository

The Doctor Clinic DevOps System is also mirrored on GitHub. The repository shows the organized folder structure including app, database, docker, k8s, scripts, terraform, and all DevOps configuration files.

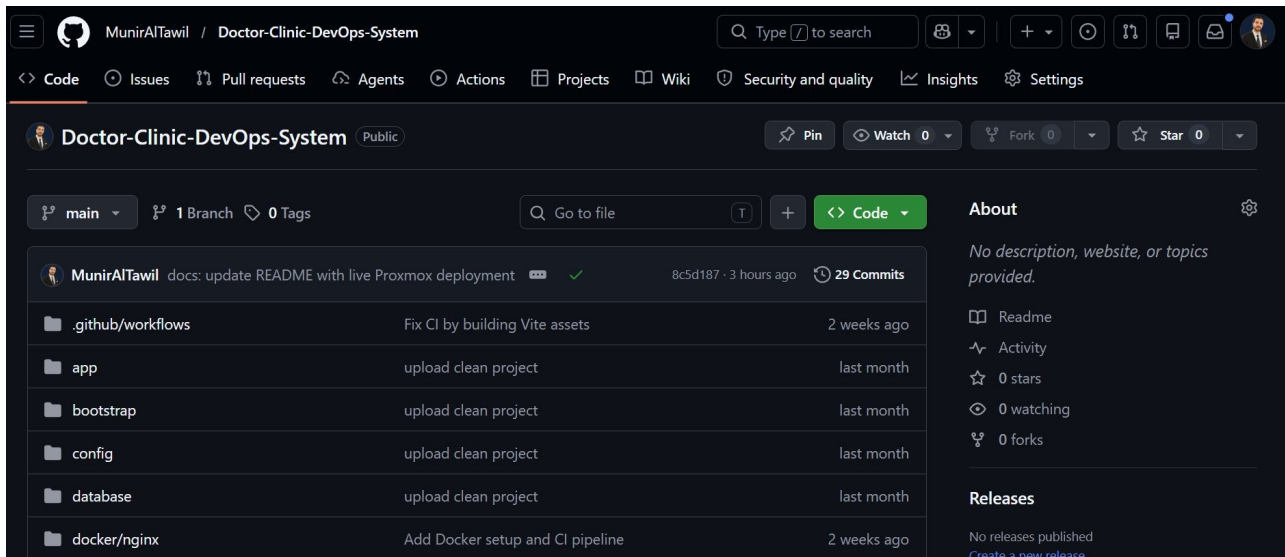


Figure 10. GitHub repository – MunirALTawil/Doctor-Clinic-DevOps-System